

Lecture 10

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- LU decomposition
- Cholesky factorization
- **Sparse matrix**

More Algorithms for Special Systems

- tridiagonal systems
- banded systems
- LU decomposition
- Cholesky factorization

Tridiagonal

A tridiagonal matrix A

$$\begin{bmatrix} d_1 & c_1 & & & & & \\ a_1 & d_2 & c_2 & & & & \\ & a_2 & d_3 & c_3 & & & \\ & & \dots & \dots & \dots & & \\ & & & a_{i-1} & d_i & c_i & \\ & & & & \dots & \dots & \dots \\ & & & & \dots & \dots & \dots \\ & & & & & a_{n-1} & d_n \end{bmatrix}$$

- storage is saved by not saving zeros
- only $n + 2(n - 1) = 3n - 2$ places are needed to store the matrix versus n^2 for the whole system
- can operations be saved? yes!

Tridiagonal

$$\begin{bmatrix} d_1 & c_1 & & & & \\ a_1 & d_2 & c_2 & & & \\ & a_2 & d_3 & c_3 & & \\ & & \dots & \dots & \dots & \\ & & & a_{i-1} & d_i & c_i \\ & & & & \dots & \dots & \dots \\ & & & & \dots & \dots & \dots \\ & & & & & a_{n-1} & d_n \end{bmatrix}$$

Start forward elimination (without any special pivoting)

- ➊ subtract a_1/d_1 times row 1 from row 2
- ➋ this eliminates a_1 , changes d_2 and does not touch c_2
- ➌ continuing:

$$d_i = d_i - \left(\frac{a_{i-1}}{d_{i-1}} c_{i-1} \right)$$

for $i = 2 \dots n$

Tridiagonal

$$\begin{bmatrix} \tilde{d}_1 & c_1 & & & & & \\ & \tilde{d}_2 & c_2 & & & & \\ & & \tilde{d}_3 & c_3 & & & \\ & & & \dots & \dots & & \\ & & & & \tilde{d}_i & c_i & \\ & & & & & \dots & \dots \\ & & & & & \dots & \dots \\ & & & & & & \tilde{d}_n \end{bmatrix}$$

This leaves an upper triangular (2-band). With back substitution:

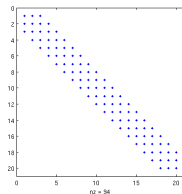
- ① $x_n = \tilde{b}_n / \tilde{d}_n$
- ② $x_{n-1} = (1/\tilde{d}_{n-1})(\tilde{b}_{n-1} - c_{n-1}x_n)$
- ③ $x_i = (1/\tilde{d}_i)(\tilde{b}_i - c_i x_{i+1})$

Tridiagonal Algorithm

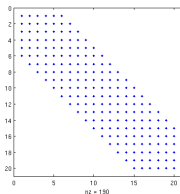
```
1 input: n, a, d, c, b
2 for i = 2 to n
3     xmult = ai-1 / di-1
4     di = di - xmult · ci-1
5     bi = bi - xmult · bi-1
6 end
7 xn = bn / dn
8 for i = n - 1 down to 1
9     xi = (bi - cixi+1) / di
10 end
```

Challenge: Will this algorithm make good use of the processor cores in a multicore processor? Why or why not?

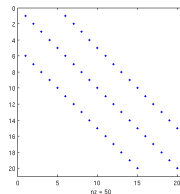
m -band



$$m = 5$$



$$m = 11$$

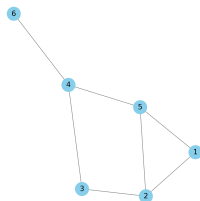


$$m = 11$$

- the m correspond to the total width of the non-zeros
- after a few passes of GE *fill-in* with occur within the band
- so an empty band costs (about) the same as a non-empty band
- one fix: reordering (e.g. Cuthill-McKee)
- generally GE will cost $\mathcal{O}(m^2n)$ for m -band systems

Motivation: Graph Theory

- Given a graph, construct associated matrix, called the graph Laplacian
- Row i of matrix corresponds to node i of graph
 - Diagonal entry is valence (total # of edges) for node i
 - Place a negative one in column j if node j is connected to i



$$\Rightarrow \begin{bmatrix} 2 & -1 & 0 & 0 & -1 & 0 \\ -1 & 3 & -1 & 0 & -1 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 3 & -1 & -1 \\ -1 & -1 & 0 & -1 & 3 & 0 \\ 0 & 0 & 0 & -1 & 0 & 1 \end{bmatrix}$$

Motivation: Graph Theory

Graph is Laplacian useful for

- Calculating spanning trees
- Partitioning a graph evenly
- and many more....

To use the graph Laplacian, you need to solve $Ax = b$ for many different vectors, b .

Motivation: Graph Theory (Multiple Right Hand Sides)

- Solve $Ax = b$ for many different b vectors
- For k different b vectors, Gaussian Elimination costs $\mathcal{O}(kn^3)$
- We can do better: LU factorization

Motivation: Symmetric Matrix

- A is symmetric, if $A = A^T$
- If $A = LU$ and A is symmetric, then could $L = U^T$?
- If so, this could save 50% of the computation of LU by only calculating L
- Save 50% of the FLOPS!
- This is achievable: LDL^T and Cholesky ($L^T L$) factorization

Factorization Methods

Factorizations are the common approach to solving $Ax = b$: simply organized Gaussian elimination.

What's next:

- LU factorization
- Cholesky factorization
- Use of the backslash operator

LU Factorization

Find L and U such that

$$A = LU$$

and L is lower triangular, and U is upper triangular.

$$L = \begin{bmatrix} 1 & 0 & \cdots & & 0 \\ \ell_{2,1} & 1 & 0 & & 0 \\ \ell_{3,1} & \ell_{3,2} & 1 & & 0 \\ \vdots & \vdots & & \ddots & \vdots \\ \ell_{n,1} & \ell_{n,2} & \cdots & \ell_{n-1,n} & 1 \end{bmatrix}$$

$$U = \begin{bmatrix} u_{1,1} & u_{1,2} & u_{1,3} & \cdots & u_{1,n} \\ 0 & u_{2,2} & u_{2,3} & \cdots & u_{2,n} \\ 0 & 0 & \ddots & \ddots & \vdots \\ \vdots & \vdots & & & u_{n-1,n} \\ 0 & 0 & & & u_{n,n} \end{bmatrix}$$

Since L and U are triangular, it is easy to apply their inverses.

Why?

- Since L and U are triangular, it is easy, $\mathcal{O}(n^2)$, to apply their inverses
- Decompose once, solve k right-hand sides quickly:
 - $\mathcal{O}(kn^3)$ with GE
 - $\mathcal{O}(n^3 + kn^2)$ with LU
- Given $A = LU$ you can compute A^{-1} , $\det(A)$, $\text{rank}(A)$, $\ker(A)$, etc...

LU Factorization

Since L and U are triangular, it is easy to apply their inverses.
Consider the solution to $Ax = b$.

$$A = LU \implies (LU)x = b$$

Regroup since matrix multiplication is associative

$$L(Ux) = b$$

Let $Ux = y$, then

$$Ly = b$$

Since L is triangular it is easy (without Gaussian elimination) to compute

$$y = L^{-1}b$$

This expression should be interpreted as “Solve $Ly = b$ with forward substitution.”

LU Factorization

Now, since y is known, solve for x

$$x = U^{-1}y$$

which is interpreted as “Solve $Ux = y$ with backward substitution.”

LU Factorization

Listing 1: LU Solve

```
1 Factor  $A$  into  $L$  and  $U$ 
2 Solve  $Ly = b$  for  $y$            use forward substitution
3 Solve  $Ux = y$  for  $x$            use backward substitution
```

LU Factorization

- If we have $Ax = b$ and perform GE we end up with

$$A = \begin{bmatrix} x & x & x & x \\ x & x & x & x \\ x & x & x & x \\ x & x & x & x \end{bmatrix} \Rightarrow \begin{bmatrix} x' & x' & x' & x' \\ 0 & x' & x' & x' \\ 0 & 0 & x' & x' \\ 0 & 0 & 0 & x' \end{bmatrix}$$

- Remember from Lectures on Sept 8 and 10, that naive Gaussian Elimination can be done by matrix multiplication

$$MAx = Mb$$

$$Ux = Mb$$

- MA is upper triangular and called U
- M is the elimination matrix

LU Factorization

As an example take one column step of GE, A becomes

$$\begin{bmatrix} 6 & -2 & 2 & 4 \\ 12 & -8 & 6 & 10 \\ 3 & -13 & 9 & 3 \\ -6 & 4 & 1 & -18 \end{bmatrix} \rightarrow \begin{bmatrix} 6 & -2 & 2 & 4 \\ 0 & -4 & 2 & 2 \\ 0 & -12 & 8 & 1 \\ 0 & 2 & 3 & -14 \end{bmatrix}$$

using the elimination matrix

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix}$$

So we have performed

$$M_1 A x = M_1 b$$

LU Factorization

From Lectures on Sept 8 and 10

- Inverting M_i is easy: just flip the sign of the lower triangular entries

$$M_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 \end{bmatrix} \Rightarrow M_1^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 1 & 0 \\ -1 & 0 & 0 & 1 \end{bmatrix}$$

- M_i^{-1} is just the multipliers used in Gaussian Elimination!
- $M_i^{-1}M_j^{-1}$ is still lower triangular, for $i < j$,
and is the union of the columns
- $M_1^{-1}M_2^{-1} \dots M_j^{-1}$ is lower triangular, with the lower triangle the multipliers from Gaussian Elimination

LU Factorization

- Zeroing each column yields another elimination matrix operation:

$$M_3M_2M_1Ax = M_3M_2M_1b$$

- $M = M_3M_2M_1$. Thus
- $L = M_1^{-1}M_2^{-1}M_3^{-1}$ is lower triangular

$$MA = U$$

$$M_3M_2M_1A = U$$

$$A = M_1^{-1}M_2^{-1}M_3^{-1}U$$

$$A = LU$$

LU (forward elimination) Algorithm

Listing 2: LU

```
1  given  $A$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5           $xmult = a_{ik}/a_{kk}$ 
6           $a_{ik} = xmult$ 
7          for  $j = k + 1 \dots n$ 
8               $a_{ij} = a_{ij} - (xmult)a_{kj}$ 
9          end
10     end
11 end
```

- U is stored in the upper triangular portion of A
- L (without the diagonal) is stored in the lower triangular

Doolittle Factorization (LU)

Listing 3: Doolittle

```
1  given  $A$ 
2  output  $L, U$ 
3
4  for  $k = 1 \dots n$ 
5       $\ell_{kk} = 1$ 
6      for  $j = k \dots n$ 
7           $u_{kj} = a_{kj} - \sum_{i=1}^{k-1} \ell_{ki} u_{ij}$ 
8      end
9      for  $j = k + 1 \dots n$ 
10          $\ell_{jk} = (a_{jk} - \sum_{i=1}^{k-1} \ell_{ji} u_{ik}) / u_{kk}$ 
11     end
12 end
```

- Mathematically the same as previous LU
- Difference is we now explicitly form L and U

What About Pivoting?

- Pivoting (that is row exchanges) can be expressed in terms of matrix multiplication
- Do pivoting during elimination, but track row exchanges in order to express pivoting with matrix P
- Let P be all zeros
 - Place a 1 in column j of row 1 to exchange row 1 and row j
 - If no row exchanged needed, place a 1 in column 1 of row 1
 - *Repeat for all rows of P*
- P is a permutation matrix
- Now using pivoting,

$$LU = PA$$

MATLAB *LU*

Like GE, *LU* needs pivoting. With pivoting the *LU* factorization always exists, even if *A* is singular. With pivoting, we get

$$LU = PA$$

```
1 >> A=rand(4,4);
2 >> b=rand(4,1);
3 >> [L,U,P]=lu(A)
4 L = 1.0000    0    0    0
5     0.9013    1.0000    0    0
6     0.0298   -0.8982    1.0000    0
7     0.7233    0.5813   -0.2670    1.0000
8 U = 0.7809    0.9890    0.4613    0.2971
9     0   -0.8838   -0.0548    0.1857
10    0    0    0.7183    0.6403
11    0    0    0    0.2065
12 P = 0    1    0    0
13     1    0    0    0
14     0    0    0    1
15     0    0    1    0
16 >> x=U\(L\(P*b))
17 x = 0.5326 0.5416 -1.2765 1.1315
18 >> A\b
19     0.5326 0.5416 -1.2765 1.1315
```

Use SYMMETRY ! YRTEMMYS esU

- Suppose

$$A = LU, \text{ and } A = A^T$$

- Then

$$LU = A = A^T = (LU)^T = U^T L^T$$

- Thus

$$U = L^{-1} U^T L^T$$

and

$$U(L^T)^{-1} = L^{-1} U^T = D$$

- We can conclude that

$$U = DL^T$$

and

$$A = LU = LDL^T$$

Symmetric Doolittle Factorization (LDL^T)

Listing 4: Symm Doolittle

```
1  given  $A$ 
2  output  $L, D$ 
3
4  for  $k = 1 \dots n$ 
5       $\ell_{kk} = 1$ 
6
7       $d_k = a_{kk} - \sum_{v=1}^{k-1} d_v \ell_{kv}^2$ 
8
9      for  $j = k + 1 \dots n$ 
10          $\ell_{kj} = 0$ 
11          $\ell_{jk} = \left( a_{jk} - \sum_{v=1}^{k-1} \ell_{jv} d_v \ell_{kv} \right) / d_k$ 
12     end
13 end
```

- Special form of the Doolittle factorization

LL^T : Cholesky Factorization

- A must be symmetric and positive definite (SPD)
- A is Positive Definite (PD) if for all $x \neq 0$ the following holds

$$x^T A x > 0$$

- Positive definite gives us an all positive D in $A = LDL^T$
 - Let $x = L^{-1}e_i$, where e_i is the i -th column of I
- L becomes $LD^{1/2}$
- $A = LL^T$, i.e. $L = U^T$
 - Half as many flops as LU !
 - Only calculate L not U

Cholesky Factorization

Listing 5: Cholesky

```
1  given  $A$   
2  output  $L$   
3  
4  for  $k = 1 \dots n$   
5       $\ell_{kk} = \left( a_{kk} - \sum_{i=1}^{k-1} \ell_{ki}^2 \right)^{1/2}$   
6  
7      for  $j = k + 1 \dots n$   
8           $\ell_{jk} = \left( a_{jk} - \sum_{i=1}^{k-1} \ell_{ji} \ell_{ki} \right) / \ell_{kk}$   
9      end  
10 end
```

Why SPD?

In general, SPD gives us

- non singular
 - If $x^T Ax > 0$, for all nonzero x
 - Then $Ax \neq 0$ for all nonzero x
 - Hence, the columns of A are linearly independent
- No pivoting
 - From algorithm, can derive that
$$|l_{kj}| \leq \sqrt{a_{kk}}$$
 - Elements of L do not grow with respect to A
 - *For short proof see book*
- Cholesky faster than LU
 - No pivoting
 - Only calculate L , not U

Why SPD?

A matrix is Positive Definite (PD) if for all $x \neq 0$ the following holds

$$x^T A x > 0$$

- For SPD matrices, use the Cholesky factorization, $A = LL^T$
- Cholesky Factorization
 - Requires no pivoting
 - Requires one half as many flops as LU factorization, that is only calculate L not L and U .
 - Cholesky will be more than *twice* as fast as LU because no pivoting means no data movement
- Use MATLAB's built-in `chol` function for routine work

Motivation Revisted

Multiple right hand sides

- Solve $Ax = b$ for k different b vectors
- Using LU factorization, the cost is $\mathcal{O}(n^3) + \mathcal{O}(kn^2)$
- Using Gaussian Elimination, the cost is $\mathcal{O}(kn^3)$

If A is symmetric

- Save 50% of the flops with LDL^T factorization
- Save 50% of the flops and many memory operations with Cholesky (L^TL) factorization

Backslash Redux

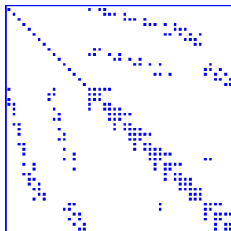
The `\` operator examines the matrix A before attempting to solve the system.

`\` is smart and does

- A triangular solve if A is triangular, or a permutation of a triangular matrix
- Cholesky factorization and triangular solves if A is symmetric and the diagonal elements of A are positive (*and* if the subsequent Cholesky factorization does not fail.)
- LU factorization if A is square and the preceding conditions are not met.
- QR factorization to obtain the least squares solution if A is not square.

Sparse Matrices

ack: Y. Saad



- Vague definition: matrix with few nonzero entries
- For all practical purposes: an $m \times n$ matrix is sparse if it has $\mathcal{O}(\min(m, n))$ nonzero entries.
- This means roughly a constant number of nonzero entries per row and column

Sparse Matrices

ack: Y. Saad

- Other definitions use a slow growth of nonzero entries with respect to n or m .
- Wilkinson's Definition: “..matrices that allow special techniques to take advantage of the large number of zero elements.” (J. Wilkinson)”
- A few applications which lead to sparse matrices: Structural Engineering, Computational Fluid Dynamics, Reservoir simulation, Electrical Networks, optimization, data analysis, information retrieval (LSI), circuit simulation, device simulation, . . .

Sparse Matrices: The Goal

- To perform standard matrix computations economically i.e., without storing the zeros of the matrix.
- For typical Finite Element /Finite difference matrices, number of nonzero elements is $\mathcal{O}(n)$.

Example

To add two square dense matrices of size n requires $\mathcal{O}(n^2)$ operations. To add two sparse matrices A and B requires $\mathcal{O}(nnz(A) + nnz(B))$ where $nnz(X)$ = number of nonzero elements of a matrix X .

remark

A^{-1} is usually dense, but L and U in the LU factorization may be reasonably sparse (if a good technique is used).

Goal

- Principle goal: *solve*

$$Ax = b$$

where $A \in \mathbb{R}^{n \times n}$, $x, b \in \mathbb{R}^n$

- Assumption: A is very sparse
- General approach: iteratively improve the solution
- Given x_0 , ultimate “correction” is

$$x_1 = x_0 + e_0$$

where $e_0 = x - x_0$, thus $Ae_0 = Ax - Ax_0$,

- or

$$x_1 = x_0 + A^{-1}r_0$$

where $r_0 = b - Ax_0$

Goal

- Principle difficulty: how do we “approximate” $A^{-1}r$ or reformulate the iteration?
- One simple idea:

$$x_1 = x_0 + \alpha r_0$$

- operation is inexpensive if r_0 is inexpensive
- requires very fast sparse mult-vec Ax_0

Sparse Matrices

- So how do we store A ?
- Fast mult-vec is certainly important; also ask
 - what type of access (rows, cols, diag, etc)?
 - dynamic allocation?
 - transpose needed?
 - inherent structure?
- unlike dense methods, not a lot of standards for iterative
 - dense BLAS have been long accepted
 - sparse BLAS still iterating
- Even data structures for dense storage not as obvious
- sparse operations have low operation/memory reference ratio

Sparse Matrix Qualification

Matrix Market attempts to classify the sparse matrix.

First Qualification (type of values and number of values):

identifier	description
Real	All entries are float
Complex	All entries are a pair of float
Integer	All entries are int
Pattern	Matrix is a pattern. Actual entries are omitted
<i>Parallel</i>	<i>Parallel structure is identified</i>

Sparse Matrix Qualification

Second Qualification (interpreting values):

identifier	description
General	A has no symmetry, no symmetry is utilized, or A is not square
Symmetric	$a_{ij} = a_{ji}$; only entries on the diagonal and below(or above) are stored.
Skew-Symmetric	$a_{ij} = -a_{ji}$; only entries below (or above) the diagonal ($= 0$) are stored.
Hermitian	$a_{ij} = \bar{a}_{ji}$; only entries on the diagonal and below (or above) are stored.

see "The Matrix Market Exchange Formats: Initial Design" by Boisvert, Pozo, Remington

Popular Storage Structures

DNS	Dense	ELL	Ellpack-Itpack
BND	Linpack Banded	DIA	Diagonal
COO	Coordinate	BSR	Block Sparse Row
CSR	Compressed Sparse Row	SSK	Symmetric Skyline
CSC	Compressed Sparse Column	BSR	Nonsymmetric Skyline
MSR	Modified CSR	JAD	Jagged Diagonal
LIL	Linked List		

note: CSR = CRS, CCS = CSC, SSK = SKS in some references

Matlab :: CSC

John R. Gilbert, Cleve, Moler and Robert Schreiber, "Sparse Matrices in MATLAB: Design and Implementation", SIAM Journal on Matrix Analysis and Applications, volume 13, number 1, pages 333–356 (1992).

$$A = \begin{bmatrix} 1.0 & 2.0 & 3.0 \\ 4.0 & 5.0 & 6.0 \\ 7.0 & 8.0 & 9.0 \end{bmatrix}$$

$$AA = [3 \quad 3 \quad 1.0 \quad 2.0 \quad 3.0 \quad 4.0 \quad 5.0 \quad 6.0 \quad 7.0 \quad 8.0 \quad 9.0]$$

- simple
- row-wise
- easy blocked formats

$$A = \begin{bmatrix} 1 & 0 & 0 & 2 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 6 & 0 & 7 & 8 & 9 \\ 0 & 0 & 10 & 11 & 0 \\ 0 & 0 & 0 & 0 & 12 \end{bmatrix}$$

$$\begin{array}{lcl} AA & = & [\quad 12.0 \quad 9.0 \quad 7.0 \quad 5.0 \quad 1.0 \quad 2.0 \quad 11.0 \quad 3.0 \quad 6.0 \quad 4.0 \quad 8.0 \quad 10.0 \quad] \\ JR & = & [\quad 5 \quad 3 \quad 3 \quad 2 \quad 1 \quad 1 \quad 4 \quad 2 \quad 3 \quad 2 \quad 3 \quad 4 \quad] \\ JC & = & [\quad 5 \quad 5 \quad 3 \quad 4 \quad 1 \quad 4 \quad 4 \quad 1 \quad 1 \quad 2 \quad 4 \quad 3 \quad] \end{array}$$

- simple, often used for entry

Question: Do you need this much storage?

$$A = \begin{bmatrix} 1 & 0 & 0 & 2 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 6 & 0 & 7 & 8 & 9 \\ 0 & 0 & 10 & 11 & 0 \\ 0 & 0 & 0 & 0 & 12 \end{bmatrix}$$

$$\begin{aligned} AA &= [\quad 1.0 \quad 2.0 \quad 3.0 \quad 4.0 \quad 5.0 \quad 6.0 \quad 7.0 \quad 8.0 \quad 9.0 \quad 10.0 \quad 11.0 \quad 12.0 \quad] \\ JA &= [\quad 1 \quad 4 \quad 1 \quad 2 \quad 4 \quad 1 \quad 3 \quad 4 \quad 5 \quad 3 \quad 4 \quad 5 \quad] \\ IA &= [\quad 1 \quad 3 \quad 6 \quad 10 \quad 12 \quad 13 \quad] \end{aligned}$$

- Length of AA and JA is nnz ; length of IA is $n + 1$
- $IA(j)$ gives the index (offset) to the beginning of row j in AA and JA (one origin due to Fortran)
- no structure, fast row access, slow column access (why?)
- related: CSC, MSR

$$A = \begin{bmatrix} 1 & 0 & 0 & 2 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 6 & 0 & 7 & 8 & 9 \\ 0 & 0 & 10 & 11 & 0 \\ 0 & 0 & 0 & 0 & 12 \end{bmatrix}$$

$$AA = [1.0 \quad 4.0 \quad 7.0 \quad 11.0 \quad 12.0 \quad * \quad 2.0 \quad 3.0 \quad 5.0 \quad 6.0 \quad 8.0 \quad 9.0 \quad 10.0]$$

$$JA = [7 \quad 8 \quad 10 \quad 13 \quad 14 \quad 14 \quad 4 \quad 1 \quad 4 \quad 1 \quad 4 \quad 5 \quad 3]$$

- places importance on diagonal (often nonzero and accessed frequently)
- first n entries are the diag
- $n + 1$ is empty
- rest of AA are the nondiagonal entries
- first $n + 1$ entries in JA give the index (offset) of the beginning of the row (the IA of CSR is in this JA)
- rest of JA are the columns indices

$$A = \begin{bmatrix} 1 & 0 & 2 & 0 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 0 & 6 & 7 & 0 & 8 \\ 0 & 0 & 9 & 10 & 0 \\ 0 & 0 & 0 & 11 & 12 \end{bmatrix} \quad \text{DIAG} = \begin{bmatrix} * & 1.0 & 2.0 \\ 3.0 & 4.0 & 5.0 \\ 6.0 & 7.0 & 8.0 \\ 9.0 & 10.0 & * \\ 11.0 & 12.0 & * \end{bmatrix} \quad \text{IOFF} = [-1 \quad 0 \quad 2]$$

- need to know the offset structure
- some entries will always be empty

$$A = \begin{bmatrix} 1 & 0 & 2 & 0 & 0 \\ 3 & 4 & 0 & 5 & 0 \\ 0 & 6 & 7 & 0 & 8 \\ 0 & 0 & 9 & 10 & 0 \\ 0 & 0 & 0 & 11 & 12 \end{bmatrix} \quad COEF = \begin{bmatrix} 1.0 & 2.0 & 0.0 \\ 3.0 & 4.0 & 5.0 \\ 6.0 & 7.0 & 8.0 \\ 9.0 & 10.0 & 0.0 \\ 11.0 & 12.0 & 0.0 \end{bmatrix} \quad JCOEF = \begin{bmatrix} 1 & 3 & 1 \\ 1 & 2 & 4 \\ 2 & 3 & 5 \\ 3 & 4 & 4 \\ 4 & 5 & 5 \end{bmatrix}$$

- Form columns from first non-zero in each row, repeat.
- used more on vector machines (what? why?)
- assumes low number of *nnz* per row (=number of columns in *COEFF* and *JCOEFF*)

JDS

- like DIA (and CDS), but more space-efficient
- costs more to gather and scatter

Take

$$A = \begin{bmatrix} 10 & -3 & 0 & 1 & 0 & 0 \\ 0 & 9 & 6 & 0 & -2 & 0 \\ 3 & 0 & 8 & 7 & 0 & 0 \\ 0 & 6 & 0 & 7 & 5 & 4 \\ 0 & 0 & 0 & 0 & 9 & 13 \\ 0 & 0 & 0 & 0 & 5 & -1 \end{bmatrix}$$

And shift:

$$A \rightarrow \begin{bmatrix} 10 & -3 & 1 \\ 9 & 6 & -2 \\ 3 & 8 & 7 \\ 6 & 7 & 5 & 4 \\ 9 & 13 \\ 5 & -1 \end{bmatrix}$$

$$A \rightarrow \begin{bmatrix} 10 & -3 & 1 & & & \\ 9 & 6 & -2 & & & \\ 3 & 8 & 7 & & & \\ 6 & 7 & 5 & 4 & & \\ 9 & 13 & & & & \\ 5 & -1 & & & & \end{bmatrix}$$

Now store the columns and column indices:

$$VAL = \begin{bmatrix} 10 & 9 & 3 & 6 & 9 & 5 \\ -3 & 6 & 8 & 7 & 13 & -1 \\ 1 & -2 & 7 & 5 & 0 & 0 \\ 0 & 0 & 0 & 4 & 0 & 0 \end{bmatrix} \quad COL = \begin{bmatrix} 1 & 2 & 1 & 2 & 5 & 5 \\ 2 & 3 & 3 & 4 & 6 & 6 \\ 4 & 5 & 4 & 5 & 0 & 0 \\ 0 & 0 & 0 & 6 & 0 & 0 \end{bmatrix}$$

JDS

- now reorder in terms of largest to smallest nnz in each row
- in JDS, the number of jagged diagonals is the number of nnz in the first row of VAL . This is the max nnz in any row.
- $PERM$: permutation array to reorder rows
- $JDIAG$: jagged diags in order
- COL : column indices
- $JDPTR$: points to the beginning of each diagonal
- advantage for mat-mat (see “Krylov subspace methods on supercomputers” by Saad)
- this is actually ITPACK or Purdue storage

$JDIAG =$ [6 9 3 10 9 5; 7 6 8 -3 13 -1; 5 -2 7 1; 4;

$COL =$ [2 2 1 1 5 5; 4 3 3 2 6 6; 5 5 4 4; 6]

$PERM =$ [4 2 3 1 5 6]

$JDPTR =$ [1 7 13 17]

$$A = \begin{bmatrix} 1.0 & 2.0 & 0.0 & 0.0 & 3.0 & 4.0 \\ 5.0 & 6.0 & 0.0 & 0.0 & 7.0 & 8.0 \\ 0.0 & 0.0 & 9.0 & 10.0 & 11.0 & 12.0 \\ 0.0 & 0.0 & 13.0 & 14.0 & 15.0 & 16.0 \\ 17.0 & 18.0 & 0.0 & 0.0 & 20.0 & 21.0 \\ 22.0 & 23.0 & 0.0 & 0.0 & 24.0 & 25.0 \end{bmatrix}$$

$$AA = \begin{bmatrix} 1.0 & 3.0 & 9.0 & 11.0 & 17.0 & 20.0 \\ 5.0 & 7.0 & 15.0 & 13.0 & 22.0 & 24.0 \\ 2.0 & 4.0 & 10.0 & 12.0 & 18.0 & 21.0 \\ 6.0 & 8.0 & 14.0 & 16.0 & 23.0 & 25.0 \end{bmatrix}$$

$$JA = [1 \quad 5 \quad 3 \quad 5 \quad 5 \quad 1 \quad 5]$$

$$IA = [1 \quad 3 \quad 5 \quad 7]$$

Blocked

- each column of AA is a 2×2 block
- $JA(k)$ = column index of $(1, 1)$ entries of the k th block
- declared as $AA(2, 2, 6)$
- blocks arise in *many* apps
- variant: variable block size

Blocked

Also row-wise

$$AA = \begin{bmatrix} 1.0 & 5.0 & 2.0 & 6.0 \\ 3.0 & 7.0 & 4.0 & 8.0 \\ 9.0 & 15.0 & 10.0 & 14.0 \\ 11.0 & 13.0 & 12.0 & 16.0 \\ 17.0 & 22.0 & 18.0 & 23.0 \\ 20.0 & 24.0 & 21.0 & 25.0 \end{bmatrix}$$

$$JA = [1 \quad 5 \quad 3 \quad 5 \quad 5 \quad 1 \quad 5]$$

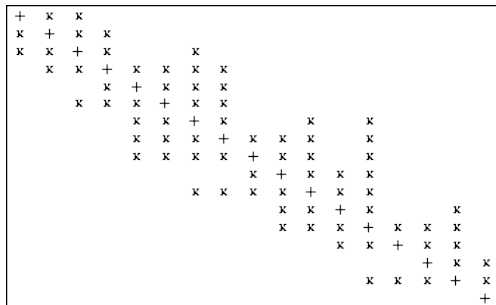
$$IA = [1 \quad 3 \quad 5 \quad 7]$$

- each row of AA is a 2×2 block (can be a drawback)
- JA , IA same, $AA(6, 2, 2)$
- if elements of blocks are accessed at the same time: rows are better (C)
- if elements of similar positions in different blocks are accessed at the same time: columns are better (C)

SSK, NSK

- for “skyline” matrices (variable band, see “Direct methods for sparse matrices” by Duff, Erisman, Reid)
- can be used for diagonal block matrices
- skyline structure is preserved in basic GE
- for symmetric: Place all the rows (in order) into *VAL*
- *IA* points to the beginning of each row
- *JA* implicit
- for nonsymmetric: store *L* in SSK format, then *U* in column-wise *SSK* see

“SPARSEKIT” by Saad



- Similar to CSR, but rather than a flat AA vector, each row is a linked list of elements
- first element of each row is accessed by $ROOT$
- each element in AA has a corresponding $NEXT$ entry
- -1 indicates the end of a row
- column lookup take $\mathcal{O}(nnz)$; one semi-costly fix: store a columnwise index in the same way as rows.
- very good element insertion time, but more memory

$$A = \begin{bmatrix} 7 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 6 & 4 \end{bmatrix}$$

- CSR
- CSC
- COO
- LIL

Example

$$A = \begin{bmatrix} 7 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 6 & 4 \end{bmatrix}$$

i	IA	JA	AA	
1	2	2	1	
2	3	4	2	
3	4	5	5	
4	2	3	2	
5	5	6	4	
6	1	1	7	
7	5	5	6	
8	3	2	2	

COO

i	IA	JA	AA	
1	1	1	7	
2	2	2	1	
3	4	3	2	
4	6	2	2	
5	7	4	2	
6	9	5	5	
7	-	5	6	
8	-	6	4	

CSR

i	IA	JA	AA	$NEXT$	
1	4	3	2	-1	
2	3	2	2	5	
3	2	2	1	1	
4	8	1	7	-1	
5	7	4	2	-1	
6	-	5	6	6	
7	-	6	4	-1	
8	-	5	5	-1	

LIL

Sparse Matrix-Vector

$$z = Ax, A_{m \times n}, x_{n \times 1}, z_{m \times 1}$$

```
1 input A, x
2 z = 0
3 for i = 1 to m
4     for col = A(i,:)
5         z(i) += A(i,col)x(col)
6     end
7 end
```

- difference between CSR and LIL is computing line 4
- CSR: rows are contiguous...(next slide)
- LIL: follow rows through linked list

Sparse Matrix-Vector

CSR

$$z = Ax, A_{m \times n}, x_{n \times 1}, z_{m \times 1}$$

```
1 DO I=1, m
2   Z(I)=0
3   K1 = IA(I)
4   K2 = IA(I+1) - 1
5   DO J=K1, K2
6     z(I) = z(I) + A(J)*x(JA(J))
7   ENDDO
8 ENDDO
```

- $\mathcal{O}(nnz)$
- marches down the rows
- very cheap

Sparse Matrix-Vector

CSR and Data Streams

$$z = Ax, A_{m \times n}, x_{n \times 1}, z_{m \times 1}$$

```
1 DO I=1, M/2
2   z(I)=0
3   z(I+M/2)=0
4   K1 = IA(I)
5   K2 = IA(I+1) - 1
6   K3 = IA(I+M/2)
7   K4 = IA(I+M/2+1) - 1
8   DO J=0, MIN(K2-K1, K4-K3)
9     z(I) = z(I) + A(K1+J)*x(JA(K1+J))
10    z(I+M/2) = z(I+M/2) + A(K3+J)*x(JA(K3+J))
11  ENDDO
12  ! ... finish up
13 ENDDO
```

- *IA* structure allows two data streams

Sparse Matrix-Matrix

- ways to optimize (“SMPP”, Douglas, Bank)

$$Z = AB, A_{m \times n}, B_{n \times p}, Z_{m \times p}$$

```
1 for i = 1 to m
2   for j = 1 to n
3     Z(i,j) = dot(A(i,:), B(:,j))
4   end
5 end
6 return Z
```

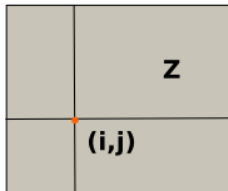
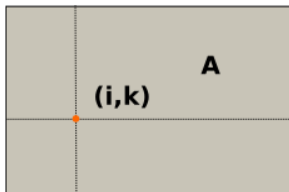
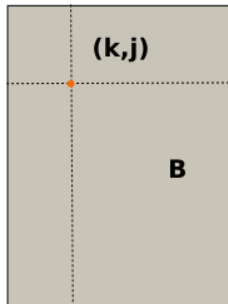
- obvious problem: column selection of B is expensive for CSR
- not-so-obvious problem: Z is sparse(!), but the algorithm doesn't account for this.

Sparse Matrix-Matrix

$$Z = AB, A_{m \times n}, B_{n \times p}, Z_{m \times p}$$

```
1 Z=0
2 for i = 1 to m
3     for colA = A(i,:)
4         for colB = A(colA,:)
5             Z(i,colB) += A(i,colA) * B(colA,colB)
6         end
7     end
8 end
9 return Z
```

- only marches down rows
- only computes nonzero entries in Z (aside from fortuitous subtractions)
- line 5 will do and insert into Z . Two options:
 - 1 precompute sparsity of Z in CSR
 - 2 use LIL for Z



Some Matlab

$$A = \begin{bmatrix} 7 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 & 0 \\ 0 & 0 & 0 & 0 & 6 & 4 \end{bmatrix}$$

i	IA	JA	AA
1	2	2	1
2	3	4	2
3	4	5	5
4	2	3	2
5	5	6	4
6	1	1	7
7	5	5	6
8	3	2	2

COO

```
1 I=[2 3 4 2 5 1 5 3 ];  
2 J=[2 4 5 3 6 1 5 2 ];  
3 A=[1 2 5 2 4 7 6 2 ];  
4  
5 A=sparse(I,J,A);
```

Some Matlab

From COO to CSC:

```
1 I=[2 3 4 2 5 1 5 3 ];  
2 J=[2 4 5 3 6 1 5 2 ];  
3 A=[1 2 5 2 4 7 6 2 ];  
4  
5 A=sparse(I,J,A);
```

Nonzeros and memory footprint:

```
1 whos A  
2 nnz(A)
```

To full and view:

```
1 B=full(A)  
2 spy(A)
```

Simple Matrix Iterations

- Solve

$$Ax = b$$

- Assumption: A is very sparse
- Let $A = N + M$, then

$$Ax = b$$

$$(N + M)x = b$$

$$Nx = b - Mx$$

- Make this into an iteration:

$$Nx_k = b - Mx_{k-1}$$

$$x_k = N^{-1}(b - Mx_{k-1})$$

- Careful choice of N and M can give effective methods
- More powerful iterative methods exist